Chapter 3: ITERATORS

This chapter is the **bridge between Containers and Algorithms**.

Without iterators:

- `sort()` ❌
- `find()` ❌
- `reverse()` ❌
- `lower_bound()` ❌
- `upper_bound()` ❌

Almost every STL algorithm works using iterators.

What is an Iterator?

Think of it as a **pointer-like object** that points to elements inside a container.

Array:

```
int arr[] = {10,20,30};
```

Pointer:

```
int* ptr = arr;
```

STL Container:

```
vector<int> v = {10,20,30};
```

Iterator:

```
vector<int>::iterator it = v.begin();
```

Why Iterators?

Different containers store data differently.

Example:

```
vector
list
set
map
deque
```

All have different internal structures.

Instead of learning separate traversal methods, STL gives:

```
iterator
```

One common way to traverse all containers.

Creating Iterator

Normal Way

```
vector<int> v={10,20,30};

vector<int>::iterator it = v.begin();
```

Modern Way (Recommended)

```
auto it = v.begin();
```

`auto` automatically detects type.

---

begin()

Returns iterator to first element.

```
vector<int> v={10,20,30};

auto it=v.begin();
```

```
cout<<*it;
```

Output:

```
10
```

Dereference Operator *

```
cout<<*it;
```

Gets value at iterator.

Example:

```
auto it=v.begin();
```

Memory:

```
10 20 30
^
it
```

```
*it
```

returns:

```
10
```

Move Iterator

Next Element

```
it++;
```

Example:

```
vector<int> v={10,20,30};

auto it=v.begin();

it++;

cout<<*it;
```

Output:

```
20
```

Jump Multiple Positions

```
it += 2;
```

Example:

```
vector<int> v={10,20,30,40,50};

auto it=v.begin();

it += 3;

cout<<*it;
```

Output:

```
40
```

---

end()

Points AFTER last element.

Example:

```
10 20 30
         ^
       end()
```

Not valid:

```
cout<<*v.end();
```

❌ Undefined Behavior

Traversing Vector

```
vector<int> v={10,20,30,40};

for(auto it=v.begin();it!=v.end();it++)
{
    cout<<*it<<" ";
}
```

Output:

```
10 20 30 40
```

Reverse Iterators

---

rbegin()

Points to last element.

```
auto it=v.rbegin();
```

---

rend()

Points before first element.

Example:

```
vector<int> v={10,20,30,40};

for(auto it=v.rbegin();it!=v.rend();it++)
{
    cout<<*it<<" ";
}
```

Output:

```
40 30 20 10
```

Iterator Functions

---

next()

Move forward.

```
auto it = next(v.begin());
```

Moves 1 step.

Move multiple:

```
auto it = next(v.begin(),3);
```

Example:

```
vector<int> v={10,20,30,40};

auto it=next(v.begin(),2);

cout<<*it;
```

Output:

```
30
```

---

prev()

Move backward.

```
auto it=prev(v.end());
```

Points to last element.

Example:

```
cout<<*prev(v.end());
```

Output:

```
40
```

Distance

Returns distance between iterators.

```
distance(first,last)
```

Example:

```
vector<int> v={1,2,3,4,5};

cout<<distance(v.begin(),v.end());
```

Output:

```
5
```

Advance

Move iterator.

```
advance(it,3);
```

Example:

```
auto it=v.begin();

advance(it,3);

cout<<*it;
```

Output:

```
4
```

Iterator Types

For interviews.

---

1. Input Iterator

Read only.

```
istream_iterator
```

Rare.

---

2. Output Iterator

Write only.

```
ostream_iterator
```

Rare.

---

3. Forward Iterator

Move only forward.

Examples:

```
forward_list
unordered_set
```

---

4. Bidirectional Iterator

Move both ways.

```
++
--
```

Examples:

```
set
map
list
```

---

5. Random Access Iterator

Most powerful.

Supports:

```
+
-
++
--
[]
```

Examples:

```
vector
deque
array
```

Interview favorite.

Iterator Invalidations

Very Important.

Example:

```
vector<int> v={1,2,3};

auto it=v.begin();

v.push_back(4);
```

Now:

```
it
```

may become invalid.

Reason:

Vector may reallocate memory.

Safe:

```
it = v.begin();
```

again.

Const Iterator

Read-only iterator.

```
vector<int>::const_iterator it=v.begin();
```

Cannot modify value.

Example:

```
*it = 100;
```

❌ Error

Common Interview Usage

Get Last Element

```
*prev(v.end())
```

Second Element

```
*next(v.begin())
```

Count Elements

```
distance(v.begin(),v.end())
```

Reverse Traverse

```
for(auto it=v.rbegin();it!=v.rend();it++)
```

Practice Questions

Easy

Q1

Print vector using iterator.

Input:

```
1 2 3 4 5
```

Output:

```
1 2 3 4 5
```

Q2

Print vector in reverse using iterator.

Q3

Print second element using iterator.

Q4

Print last element using:

```
prev()
```

Medium

Q5

Find distance between:

```
begin()
end()
```

Q6

Move iterator to 4th element using:

```
advance()
```

Print it.

Q7

Find middle element using iterator.

Hint:

```
advance()
distance()
```

Q8

Print every alternate element.

Input:

```
1 2 3 4 5 6
```

Output:

```
1 3 5
```

Interview Questions

Q9

Delete element pointed by iterator.

Use:

```
erase(it)
```

Q10

Insert value before iterator.

Use:

```
insert(it,value)
```

Q11

Erase all even numbers using iterator traversal.

Revision Sheet

Create

```
auto it=v.begin();
```

Access

```
*it
```

Move

```
it++
it--
it+=k
```

(Random Access only)

Special Functions

```
next(it)
prev(it)

advance(it,k)

distance(a,b)
```

Begin/End

```
begin()
end()
```

Reverse

```
rbegin()
rend()
```

Types

```
Input
Output
Forward
Bidirectional
Random Access
```

Most Important Interview Concepts

```
Iterator
Invalidation
next()
prev()
distance()
advance()
```

Mini STL Challenge

Input:

```
1 2 3 4 5 6 7 8 9
```

Tasks:

1. Print using iterator.

2. Print reverse using reverse iterator.
3. Print second element.
4. Print last element.
5. Print middle element.
6. Print distance between begin and end.
7. Move iterator to 5th element and print.
8. Delete 4th element using iterator.
9. Insert 100 before 3rd element.
10. Print final vector.
